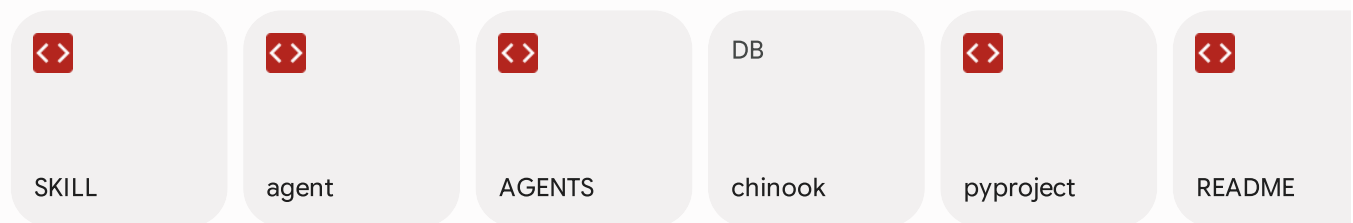


Analyze and understand this project and explain each concept used in details

The attached zip contains too many files to process. For more information, see [here](#).



Analyze and understand this project and explain each concept used in details

This project is a **Text-to-SQL Deep Agent**—an advanced, autonomous Python application designed to translate natural language questions into executable SQL queries, run them against a local database, and return the analyzed results in plain text.

Based on the files provided, the project is built using LangChain, the Deep Agents framework, and a local Large Language Model (LLM).

Here is a detailed breakdown of the core concepts and architectures driving this project.

1. The "Deep Agents" Framework

At its core, this project uses a specialized framework called `deepagents` (which is built on top of LangGraph). Standard LLM agents can struggle with complex tasks because their "context window" (their memory of the conversation) fills up with intermediate steps, causing them to forget the original goal or hallucinate.

- **Context Management:** Deep Agents solves this by managing how much information is fed back to the LLM at once, preventing context overflow during complex database explorations.

- **Filesystem Backend:** You'll notice in `agent.py` and `README.md` the mention of a `FilesystemBackend`. This gives the agent a "scratchpad" to save intermediate results or state, allowing it to work systematically rather than holding everything in its immediate memory.
- **Task Planning:** The agent uses a tool called `write_todos`. When faced with a complex query (like joining multiple tables), it is instructed to write out a step-by-step plan before writing the code.

2. Local LLM Integration (Ollama)

The `agent.py` script orchestrates the logic, and notably, it instantiates the LLM using `ChatOllama(model="gemma4:e2b")`.

- **Local Execution:** This is a powerful pattern for data security. By running the model locally via Ollama within the Python environment, no database schemas or data are sent to external APIs (like OpenAI or Anthropic).
- **Zero Temperature:** The script explicitly sets `temperature=0`. LLMs are inherently creative, which is great for writing poetry but terrible for writing SQL syntax. A temperature of zero forces the model to be entirely deterministic, ensuring consistent, syntactically valid SQL queries.
- **Large Context Window:** `num_ctx=32192` ensures the model has enough short-term memory to ingest the entire database schema and the instructions without truncating vital information.

3. Database Introspection & Tooling (SQLAlchemy)

The agent doesn't just guess table names; it actively investigates the database before writing queries.

- **The Chinook Database:** The project uses `chinook.db`, a classic sample SQLite database representing a digital media store (artists, albums, tracks, invoices).
- **SQLDatabaseToolkit:** LangChain provides this toolkit, giving the LLM specific "functions" it can call. For example, the agent can call `sql_db_schema` to pull the exact column names and foreign keys of the `Invoice` table before attempting a `JOIN` operation.

4. Prompt Engineering & Guardrails

The `.md` files (`AGENTS.md` and `SKILL.md`) act as the "system instructions" or personality of the agent. They establish strict behavioral guardrails:

- **Read-Only Safety:** The agent is explicitly forbidden from running destructive commands (`INSERT`, `UPDATE`, `DROP`, `ALTER`). It is restricted to `SELECT` statements.
- **Query Optimization:** It is instructed to limit results to 5 rows by default, sort meaningfully, and avoid `SELECT *`. This mimics how one might optimize complex queries to avoid overwhelming a dashboard or reporting tool.

- **Error Recovery:** If a SQL query fails (e.g., a syntax error or hallucinated column), the framework captures the SQLite error and feeds it back to the LLM. The instructions guide the agent to read the error, re-examine the schema, and rewrite the query autonomously.

5. Observability (LangSmith)

The `README.md` details integration with LangSmith. When building autonomous agents, it is very difficult to know *why* an agent gave a specific answer. LangSmith acts as an x-ray for the application, tracing the exact sequence of events: the user's prompt, the agent's plan, the tools it called (like schema lookups), the exact SQL it generated, and the raw data returned from the database.

Would you like to dive deeper into the code mechanics of how the Python `agent.py` script ties these components together, or would you prefer to explore how the `write_todos` planning tool functions under the hood?

what is FilesystemBackend and write_todos sql_db_schema and where it is used
where sql query fails detection or hallucination capture used in code
also explain langsmith in details

Here is a detailed breakdown of these specific concepts, tools, and mechanics based on the code and documentation provided in your project.

1. The Core Tools: `FilesystemBackend`, `write_todos`, and `sql_db_schema`

These three components form the "hands and memory" of your agent, allowing it to interact with the world outside its prompt.

`FilesystemBackend`

- **What it is:** A feature of the `deepagents` framework that provides the agent with a local, persistent workspace (a directory on your hard drive) rather than just transient memory.
- **Where it's used in your code:** It is imported in `agent.py` (`from deepagents.backends import FilesystemBackend`). As noted in the `README.md` snippet, it is initialized to use the base directory (`root_dir=base_dir`).

- **Why it matters:** Standard LLMs forget things as conversations get long. By giving the agent a filesystem backend, it can save large chunks of data (like complex database schemas or intermediate query results) to physical files and read them later, keeping its active "context window" clean and focused.

write_todos

- **What it is:** A planning tool (a callable function) provided to the agent to enforce "Chain of Thought" reasoning.
- **Where it's used:** It is heavily referenced in the instruction files (`AGENTS.md` and `SKILL.md`). In `SKILL.md` under "Workflow for Complex Queries", the agent is strictly instructed to use `write_todos` to break down tasks (e.g., "Identify all tables needed", "Map relationships").
- **Why it matters:** If an LLM tries to write a complex, 4-table SQL `JOIN` immediately, it often hallucinates or makes logical errors. By forcing it to use `write_todos` first, the agent writes out a step-by-step plan. This plan becomes part of its context, acting as a logical blueprint that drastically improves the accuracy of the final SQL query.

sql_db_schema

- **What it is:** A specialized tool belonging to LangChain's `SQLDatabaseToolkit` .
- **Where it's used:** The toolkit is initialized in `agent.py` and passed to the agent. `SKILL.md` explicitly tells the agent: *"Use `sql_db_schema` for EACH table to find join columns."*
- **Why it matters:** Instead of guessing column names (which causes hallucinations), the agent passes a table name (like "Invoice") into this tool. The tool queries the SQLite database's metadata and returns the exact column names, data types, and foreign key relationships back to the LLM.

2. Where is SQL Query Failure Detection / Hallucination Capture Used?

You won't find a massive `try/except` block written manually in `agent.py` for query failures. That is because the error detection is elegantly handled by a combination of the **LangChain Toolkit** and your **Prompt Engineering (SKILL.md)**.

Here is how the "capture and fix" loop actually happens under the hood:

1. **The Execution (LangChain):** When the agent generates a SQL query and uses the `sql_db_query` tool to run it, LangChain executes it against `chinook.db` using SQLAlchemy.
2. **The Capture (The Toolkit):** If the LLM hallucinates a column name (e.g., `SELECT user_name FROM Customer`), SQLite throws an `OperationalError` . The LangChain toolkit *catches* this Python exception. Instead of crashing your `agent.py` script, the tool

takes the exact error string (e.g., *"Error: no such column: user_name"*) and returns it to the agent as the tool's output.

3. **The Correction (SKILL.md):** The agent reads this error message. Because of the **"Error Recovery"** section you defined in `SKILL.md`, the agent knows exactly what to do:

"If a query fails... 2. Syntax error — Re-examine JOINS... 1. Empty results — Verify column names..."

4. **The Loop:** The agent autonomously realizes its mistake, calls `sql_db_schema` again to find the correct column name (e.g., `FirstName`), rewrites the query, and tries again. All of this happens in a hidden loop before the user ever sees the final answer.

3. Explaining LangSmith in Detail

As highlighted in your `README.md`, LangSmith is integrated into this project via environment variables (`LANGCHAIN_TRACING_V2=true`).

What is LangSmith?

LangSmith is an observability platform designed specifically for LLM applications. Think of it as an "X-ray machine" or a deeply advanced debugger for AI agents.

Why is it critical for this specific project?

When you run `agent.py` and ask, *"Which employee generated the most revenue?"*, the console might just print the final answer 15 seconds later. But during those 15 seconds, the agent is essentially "talking to itself"—calling tools, writing plans, making mistakes, and fixing them. LangSmith makes this invisible process visible.

In LangSmith, you will see a **Trace** (as shown in your uploaded image `text-to-sql-langsmith-trace.jpg`), which provides:

- **The Execution Tree:** A visual, chronological tree of every step the agent took. You can click on the `write_todos` step to see exactly what plan the agent came up with.
- **Tool Inputs and Outputs:** You can see the exact SQL query the LLM sent to the database, and the exact raw data (like a JSON array of rows) the database sent back.
- **Error Visibility:** If the agent made a syntax error on its first try and fixed it on the second try, LangSmith logs the error. This is crucial for developers; if you notice the agent constantly failing at `JOIN` statements, you know you need to update `SKILL.md` to give it better instructions on how to write joins.
- **Token Tracking:** LLMs process text in "tokens." Because this agent uses a local model (`gemma4:e2b` via Ollama) with a massive context window (`num_ctx=32192`), LangSmith tracks exactly how many tokens were used in the prompt and the completion, helping you optimize performance and latency.



agent

provide d2 diagram explanation of end2end working of this code

Here is the D2 diagram representing the end-to-end architecture and workflow of your `agent.py` script.

You can render this diagram by pasting the code block below into the official D2 playground at play.d2lang.com or by using the D2 CLI.

D2 Diagram Code

Code snippet



```
direction: right
```

```
User: User {  
  shape: person  
}
```

```
CLI: agent.py (CLI interface) {  
  shape: component  
  style: { fill: "#f4f4f4" }  
}
```

```
Agent_Core: Deep Agent System {  
  shape: package  
  style: { fill: "#e1f5fe" }
```

```
  Planner: write_todos tool  
  Toolkit: SQLiteDatabaseToolkit  
  Memory: FileSystemBackend  
}
```

```
LLM: ChatOllama\n(gemma4:e2b) {  
  shape: component  
  style: { fill: "#e8f5e9" }  
}
```

```
Database: chinook.db\n(SQLite) {
```

```

    shape: cylinder
    style: { fill: "#fff3e0" }
  }

Prompts: AGENTS.md &\nSKILL.md {
  shape: note
}

# The Execution Flow
User -> CLI: 1. Asks Natural Language Question
CLI -> Agent_Core: 2. Parses Args & Invokes Agent

Prompts -> Agent_Core: Loads Guardrails/Instructions
Agent_Core -> LLM: Connects to Local Model (Temp=0)

Agent_Core.Planner -> LLM: 3. Formulates Step-by-Step Plan
Agent_Core.Toolkit -> Database: 4. Introspects Schema (sql_db_schema)
Database -> Agent_Core.Toolkit: Returns Tables/Columns

Agent_Core.Toolkit -> Database: 5. Executes SQL (sql_db_query)
Database -> Agent_Core.Toolkit: Returns Data Rows

Agent_Core.Memory -> Agent_Core.Memory: 6. Caches Intermediate Results

Agent_Core -> LLM: 7. Passes Data for Synthesis
LLM -> Agent_Core: Returns Natural Language Answer

Agent_Core -> CLI: 8. Yields Final Answer & Logprobs
CLI -> User: 9. Displays Formatted Rich Output

```

Step-by-Step Explanation of the Flow

Here is how the code practically executes this diagram from end to end when you run `python agent.py "How many customers are from Canada?"`.

Phase 1: Setup and Initialization (Steps 1-2)

- The Entry Point:** The script uses `argparse` to capture the user's natural language question from the command line. It uses the `rich` library to print this question to the terminal nicely.
- Environment & Connections:** The `create_sql_deep_agent()` function is called. It locates the local SQLite database (`chinook.db`). It initializes `ChatOllama` to connect to your local `gemma4:e2b` model, explicitly setting `temperature=0` so the model prioritizes precise SQL over creative text.

Phase 2: Agent Tooling and Context Loading

- Equipping the Agent:** The agent is built by combining several critical components:

- **The Toolkit:** LangChain's `SQLDatabaseToolkit` is instantiated. This creates specific Python functions the LLM is allowed to call (like `sql_db_schema` to read the database structure and `sql_db_query` to run SQL).
- **The Planner:** The `write_todos` tool is injected.
- **The Instructions:** The text from `AGENTS.md` and `SKILL.md` is loaded. This acts as the agent's "system prompt," telling it to read schemas before writing queries and forbidding destructive commands (like `DROP` or `UPDATE`).
- **The Memory:** `FilesystemBackend` is attached, giving the agent a local folder to write down notes if the context gets too large.

Phase 3: The Deep Agent Execution Loop (Steps 3-7)

4. **Planning:** Once `agent.invoke()` is called, the LLM receives the question. Based on the instructions, it first calls the `write_todos` tool to output a logical plan (e.g., *1. Look up Customer table schema. 2. Write a COUNT query with a WHERE clause for Canada.*).
5. **Introspection:** The agent calls the `sql_db_schema` tool. The toolkit executes a metadata lookup against `chinook.db` and returns the exact column names (like `Country`, `CustomerId`) back to the LLM.
6. **Execution & Auto-Correction:** The LLM generates a SQL string and calls `sql_db_query`.
 - If the query is correct, the toolkit returns the data rows.
 - If the LLM makes a syntax error, the toolkit catches the SQLite error and feeds the error string back to the LLM. The LLM reads the error and loops back to fix its own query before proceeding.

Phase 4: Synthesis and Output (Steps 8-9)

7. **Final Synthesis:** The LLM takes the raw database rows (e.g., `[(8,)]`) and synthesizes them into a human-readable answer ("There are 8 customers from Canada.").
8. **Logging & Display:** The script intercepts the final response object. It extracts the `logprobs` (a statistical measure of the model's confidence in the words it chose) and prints the top 5 to the terminal.
9. **Result:** Finally, the script extracts the plain text answer and wraps it in a formatted, green-bordered `rich.Panel` for the user to read.

Errors

```
err: failed to compile -: -:8:10: unknown shape "component"
err: -:22:10: unknown shape "component"
```


err: -:32:10: unknown shape "note"

Ah, that's my mistake! I used `component` and `note`, which are standard in PlantUML and Mermaid, but D2 doesn't officially support them in its core shape library.

We can fix this instantly by swapping those out for D2's native `rectangle` and `document` shapes.

Here is the corrected, compilable D2 code:

Code snippet



```
direction: right

User: User {
  shape: person
}

CLI: agent.py (CLI interface) {
  shape: rectangle
  style: { fill: "#f4f4f4" }
}

Agent_Core: Deep Agent System {
  shape: package
  style: { fill: "#e1f5fe" }

  Planner: write_todos tool
  Toolkit: SQLiteDatabaseToolkit
  Memory: FilesystemBackend
}

LLM: ChatOllama\n(gemma4:e2b) {
  shape: rectangle
  style: { fill: "#e8f5e9" }
}

Database: chinook.db\n(SQLite) {
  shape: cylinder
  style: { fill: "#fff3e0" }
}

Prompts: AGENTS.md &\nSKILL.md {
  shape: document
}

# The Execution Flow
```

```
User -> CLI: 1. Asks Natural Language Question
CLI -> Agent_Core: 2. Parses Args & Invokes Agent

Prompts -> Agent_Core: Loads Guardrails/Instructions
Agent_Core -> LLM: Connects to Local Model (Temp=0)

Agent_Core.Planner -> LLM: 3. Formulates Step-by-Step Plan
Agent_Core.Toolkit -> Database: 4. Introspects Schema (sql_db_schema)
Database -> Agent_Core.Toolkit: Returns Tables/Columns

Agent_Core.Toolkit -> Database: 5. Executes SQL (sql_db_query)
Database -> Agent_Core.Toolkit: Returns Data Rows

Agent_Core.Memory -> Agent_Core.Memory: 6. Caches Intermediate Results

Agent_Core -> LLM: 7. Passes Data for Synthesis
LLM -> Agent_Core: Returns Natural Language Answer

Agent_Core -> CLI: 8. Yields Final Answer & Logprobs
CLI -> User: 9. Displays Formatted Rich Output
```

This will compile cleanly for you now!

how I can trace and check what agent is doing in background while running

In this specific project, there are three primary ways to look under the hood and trace exactly what the agent is thinking, planning, and executing in the background.

Here is how you can monitor its background activities:

1. The Primary Method: LangSmith Tracing (The "X-Ray")

The absolute best way to see what the agent is doing is by using LangSmith, which is already integrated into your project. LangSmith provides a visual dashboard of the entire execution tree.

How to set it up:

Based on your `README.md`, you need to enable it via your environment variables. Open or create a `.env` file in the same directory as `agent.py` and add the following:

Bash



```
LANGCHAIN_TRACING_V2=true
LANGSMITH_ENDPOINT=https://api.smith.langchain.com
LANGCHAIN_API_KEY=your_langsmith_api_key_here
LANGCHAIN_PROJECT=text2sql-deepagent
```

(You can get a free API key by signing up at smith.langchain.com)

What it shows you:

Once enabled, every time you run `python agent.py`, you can log into LangSmith and see:

- **The exact sequence of tool calls:** You will see when it called `write_todos` and what plan it wrote.
- **Database Queries:** You will see the exact SQL query it sent to `sql_db_query` and the raw JSON data that `chinook.db` sent back.
- **Hidden Errors:** If the agent wrote a bad SQL query, you will see the SQLite error it triggered and how the agent corrected itself on the next try.
- **Token Usage:** It tracks how many tokens the `gemma4:e2b` model consumed for the prompt and completion.

2. Inspecting the Filesystem Backend (The "Scratchpad")

Because your `agent.py` uses the `FilesystemBackend`, the agent has a physical workspace on your hard drive where it saves its thoughts and intermediate work.

In `agent.py`, the backend is initialized like this:

```
backend=FilesystemBackend(root_dir=base_dir)
```

How to check it:

When you run a complex query, check your project folder (the same folder where `agent.py` lives). You should see new files or directories generated by the agent. If the agent uses the `write_todos` tool to plan a complex JOIN operation, it will physically write that plan into a text file in this directory. Opening these files allows you to read the agent's "scratch notes" in real-time.

3. Modifying `agent.py` for Terminal Debugging

Currently, `agent.py` prints a nice, formatted UI using the `rich` library, showing the final answer and some token probabilities (`logprobs`). If you want to see the raw background chatter directly in your terminal without using LangSmith, you can enable LangChain's global debug mode.

You can add this to the very top of `agent.py` (right after your imports):

Python



```
import langchain  
langchain.debug = True
```

What this does:

When you run the script, your terminal will explode with green and blue text showing every single prompt sent to Ollama, every tool invocation, and every raw response. It is much messier than LangSmith, but it provides an immediate, localized view of the background processes.

